



MÓDULO 4 · T13

Modal en React Native

 *Modales con backdrop y detalle Pokémon*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión aprenderás a usar el componente `<Modal>` de React Native para mostrar información detallada sobre un Pokémon seleccionado. Al terminar, serás capaz de implementar patrones de galería-detalle con backdrop interactivo.

1

Usar `<Modal>`

Integrar el componente `<Modal>` de React Native con visibilidad controlada por estado.

2

Backdrop interactivo

Implementar un backdrop que cierre el modal al tocarlo, usando doble `<Pressable>`.

3

Pasar datos al modal

Guardar el Pokémon seleccionado en estado y pasarlo al modal para mostrar su detalle.

4

Patrones galería → detalle

Aplicar el patrón común de lista con tap que abre un modal con información ampliada.

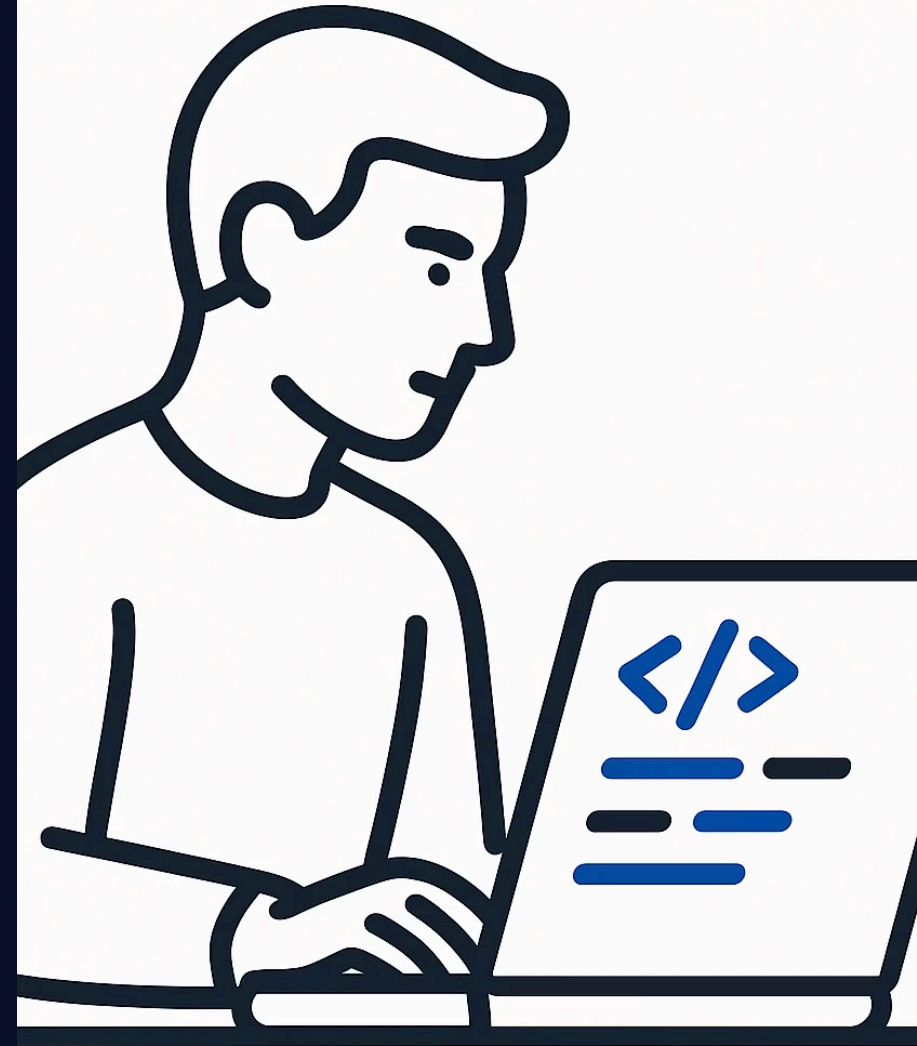
Modal básico

El componente `<Modal>` de React Native se controla con la prop `visible`. Con `transparent` el fondo nativo desaparece y `animationType` define cómo entra y sale. El prop `onRequestClose` es imprescindible en Android para gestionar el botón físico de atrás.

```
const [visible, setVisible] = useState(false);

<Modal
  visible={visible}
  transparent
  animationType="fade"
  onRequestClose={() => setVisible(false)} // botón atrás Android
>
  <View style={styles.contenido}>
    <Text>Contenido del modal</Text>
    <Button
      title="Cerrar"
      onPress={() => setVisible(false)}
    />
  </View>
</Modal>
```

 Con `transparent` activo, el fondo del modal es completamente transparente – tú defines el aspecto visual mediante estilos propios.



Backdrop y doble Pressable

¿Por qué doble Pressable?

El `<Pressable>` exterior actúa como backdrop: ocupa toda la pantalla y cierra el modal al tocarlo. El `<Pressable>` interior envuelve la card y captura el toque con `onPress={() => {}}`, evitando que el evento suba al backdrop.

- ✓ Este patrón garantiza que tocar fuera de la card cierre el modal, pero tocar dentro no lo haga – UX estándar en apps nativas.

Implementación

```
<Modal
  visible={selected !== null} transparent
  animationType="fade" onRequestClose={handleClose}>
  /* Backdrop: toca para cerrar */
  <Pressable style={styles.backdrop} onPress={handleClose}>
    /* Card: evita que el toque
    en la card cierre el modal */
    <Pressable style={styles.card} onPress={() => {}}>
      {selected && <DetallePokemon pokemon={selected} />}
    </Pressable>
  </Pressable>
</Modal>
```

Abrir el modal con datos

El estado `selected` almacena el Pokémon pulsado o `null` cuando el modal está cerrado. Los callbacks `handleSelect` y `handleClose` se memorizan con `useCallback` para evitar renders innecesarios en los items de la lista.

```
const [selected, setSelected] = useState<Pokemon | null>(null);

const handleSelect = useCallback((p: Pokemon) => setSelected(p), []);
const handleClose = useCallback(() => setSelected(null), []);

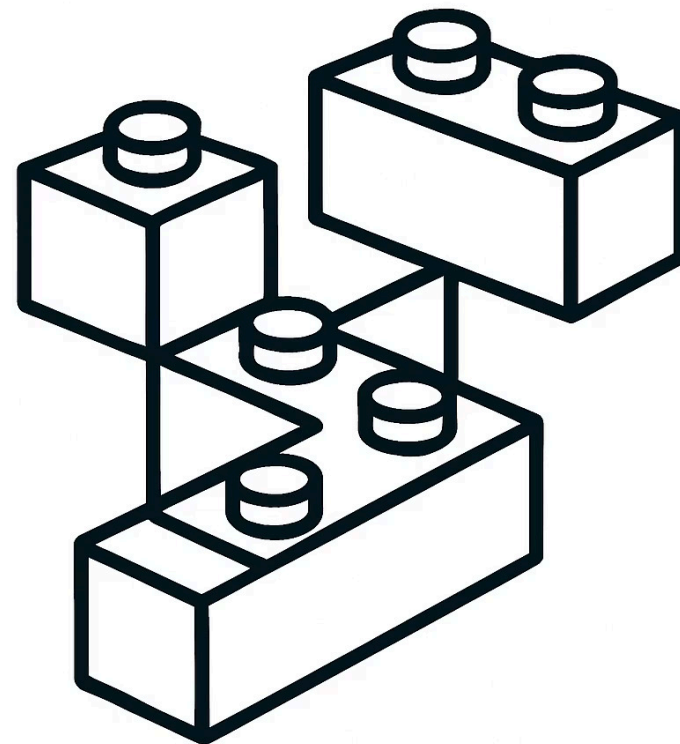
// En la lista:
<FlatList
  data={pokemons}
  renderItem={({ item }) => (
    <TouchableOpacity onPress={() => handleSelect(item)}>
      <Text>{item.nombre}</Text>
    </TouchableOpacity>
  )}
/>
```

animationType

La prop `animationType` define cómo entra y sale el modal. La combinación de `'fade'` con un backdrop semitransparente es el estándar visual de las apps modernas: suave, no intrusiva y elegante.

Valor	Efecto
<code>'none'</code>	Sin animación – aparece y desaparece instantáneamente
<code>'fade'</code>	Aparece con fundido – transición suave de opacidad
<code>'slide'</code>	Sube desde abajo – típico de sheets y menús inferiores

❗ `'fade'` + backdrop semitransparente = look de app moderna. Es la combinación recomendada para modales de detalle.



Estilos del backdrop y card

El backdrop ocupa toda la pantalla con `flex: 1` y centra la card con `alignItems / justifyContent: 'center'`. El color `rgba(0,0,0,0.5)` crea el efecto de atenuación del fondo. La card usa `elevation` para proyectar sombra en Android y reforzar la sensación de profundidad.

```
const styles = StyleSheet.create({
  backdrop: {
    flex: 1,
    backgroundColor: 'rgba(0, 0, 0, 0.5)', // semitransparente
    alignItems: 'center',
    justifyContent: 'center',
  },
  card: {
    backgroundColor: '#fff',
    borderRadius: 24,
    padding: 24,
    alignItems: 'center',
    width: 280,
    // sombra para elevación visual
    elevation: 12,
  },
});
```

📌 **Buena práctica:** usa `StyleSheet.create` para los estilos estáticos del backdrop y la card – React Native los optimiza internamente.

PokeApp — PokedexScreen.tsx

Este es el componente completo tal y como aparece en la aplicación de ejemplo. El modal reutiliza el componente `DetallePokemon` que incluye la flip animation desarrollada en el T12, integrando de forma natural los contenidos del módulo anterior.

```
<Modal
  visible={selected !== null}
  transparent
  animationType="fade"
  onRequestClose={handleClose}
>
  <Pressable
    style={styles.backdrop}
    onPress={handleClose}
  >
    <Pressable
      style={styles.container}
      onPress={() => {}}
    >
      {selected &&
        <DetallePokemon pokemon={selected} />}
    </Pressable>
  </Pressable>
</Modal>
```

❗ El modal muestra la tarjeta con la **flip animation del T12**, conectando el contenido de ambas sesiones en una sola pantalla.

Tour starter/PokemonModal.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica de la pantalla con tres puntos marcados como `TODO` que deberás completar durante el ejercicio: el estado, el handler de selección y el propio `<Modal>`.

```
export default function PokemonModal() {  
  // TODO: const [selected, setSelected] = useState<Pokemon | null>(null);  
  
  return (  
    <FlatList  
      data={POKEMONS}  
      renderItem={({ item }) => (  
        <Pressable  
          onPress={() => {  
            /* TODO: setSelected(item) */  
          }}  
        >  
          <Text>{item.nombre}</Text>  
        </Pressable>  
      )}  
    </FlatList>  
    // TODO: Modal aquí  
  );  
}
```

⚠ No modifiques los estilos base del `StyleSheet` – céntrate en los tres `TODO` marcados arriba.

Ejercicio T13

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el patrón de detalle visual en modales.

1 Abre el fichero de inicio

Navega a `starter/PokemonModal.tsx` en tu editor.

2 Añade el estado `selected`

Declara `const [selected, setSelected] = useState<Pokemon | null>(null)`.

3 Conecta el handler al `FlatList`

Al pulsar un item, guarda el Pokémon en `selected` mediante `setSelected(item)`.

4 Implementa el `<Modal>`

Añade el `<Modal>` con backdrop semitransparente y doble `<Pressable>`.

5 Bonus: imagen oficial-artwork

Añade la imagen `official-artwork` del Pokémon seleccionado en la card del modal.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá implementar cualquier flujo galería-detalle en React Native con modales nativos, accesibles y visualmente pulidos.

Concepto	Sintaxis clave	Para qué sirve
<code><Modal visible={bool}></code>	<code>visible={selected !== null}</code>	Se muestra cuando visible es true
<code>transparent</code>	prop booleana en Modal	Fondo del modal transparente
<code>animationType</code>	<code>'fade' 'slide' 'none'</code>	Define la animación de entrada/salida
<code>onRequestClose</code>	<code>onRequestClose={handleClose}</code>	Botón atrás en Android
Doble <code><Pressable></code>	Backdrop exterior + card interior	Backdrop cierra, card no
<code>rgba(0,0,0,0.5)</code>	<code>backgroundColor: 'rgba(0,0,0,0.5)'</code>	Fondo semitransparente del backdrop

✅ ¡Enhorabuena! Ahora sabes implementar modales nativos con backdrop, animación y paso de datos en cualquier app React Native. 🚀